



Rapport Travail 10 - Group Dih

REY Dorian

DURAND Eliot

BASILE Francesco-Pio

BOUREUX Nathan

Faculté des Sciences
Université de Montpellier

Avril 2026

Table des matières

1	Entraînement du modèle DistilBERT	3
1.1	Fine tuning sur un dataset uniquement composé de CVE Java	3
1.2	Comparaison de prédiction sur de réel CVSS entre le modèle full et le modèle only java ..	4
1.3	Test de notre modèle sur des descriptions inventées	4
1.4	Conclusion	6
1.5	Amélioration du code Google Colab pour la réutilisation	6
2	Les tests sur les différents sources de l'agrégateur	7
2.1	Résumé des performances par source	7
2.2	Précision et Rappel (Consensus « Soft Ground Truth »)	7
2.3	Conclusion de l'analyse	8
3	Simplification du code frontend	8
4	Objectif pour la séance suivante	8
5	Annexe	9

1 Entraînement du modèle DistilBERT

1.1 Fine tuning sur un dataset uniquement composé de CVE Java

Comme pour ces dernières semaines, tous les résultats d'entraînement sont disponibles à la Fig. 1. Le code colab est disponible à la Fig. 2. Le meilleur modèle actuel est disponible à la Fig. 3.

Cette semaine, le but est d'améliorer le modèle grâce, à partir d'un clone de notre plus gros modèle, de l'entraîner uniquement sur des CVSS touchant à l'écosystème Java. Après une légère modification du script python afin de le rendre modulaire pour la sélection ou non d'écosystème précis, le dataset fait 8500 lignes. Cela peut paraître léger mais pour notre modèle préentraîné cela est suffisant, en plus de rappeler que les données sont de bonnes qualité.

Ci-dessous les résultats du fine tuning :

Métrique	E1	E2	E3	...	E10	E11	E12	Modèle full
Accuracy Train	0.9088	0.9165	0.9205	...	0.9437	0.9457	0.9481	0.9313
Accuracy Val.	0.9197	0.9211	0.9219	...	0.9322	0.9329	0.9334	0.9230
F1 Train	0.9057	0.9142	0.9184	...	0.9431	0.9450	0.9475	0.9297
F1 Val.	0.9171	0.9186	0.9199	...	0.9313	0.9323	0.9326	0.9214
Loss Train	2.0339	1.8438	1.7392	...	1.2115	1.1872	1.1150	1.5413
Loss Val.	1.7595	1.6906	1.6466	...	1.4751	1.4604	1.4652	1.7370

Tableau 1. – Entraînement sur un dataset contenant uniquement des CVE Java

Beaucoup d'epochs ici car afin de préserver les poids du modèle, le learning rate est très bas, ce qui explique qu'il progresse « lentement ». L'epoch 11 est la meilleure, car elle affiche la val loss la plus basse, et l'epoch 12 montre les premiers signes de sur-apprentissage.

Par rapport au modèle « full », qui était le meilleur, on voit une augmentation dans chaque métrique. L'accuracy validation passe les 93%, la F1 également et la val loss descend de presque 0,3, symbole d'une meilleure stabilité. Notre modèle est donc encore meilleur, nous l'appellerons « only java ».

1.2 Comparaison de prédiction sur de réel CVSS entre le modèle full et le modèle only java

CVE ID	Score NVD (Réel)	Score Full Model	Score Model Only Java
CVE-2024-22288	7.1	6.1	6.1
CVE-2024-27073	5.5	5.5	5.5
CVE-2024-46591	7.5	7.5	7.5
CVE-2025-5799	8.8	8.8	8.8
CVE-2025-4394	6.8	6.1	6.1
CVE-2025-10442	6.3	9.8	8.8
CVE-2026-23072	5.5	5.5	5.5
CVE-2026-0007	7.8	7.8	7.8
CVE-2026-1663	4.3	4.3	4.3

Tableau 2. – Comparaison des scores CVSS entre les données réelles (NVD), le modèle baseline (temporel) et le modèle final (full dataset)

Le modèle only java est meilleur sur la CVE-2025-10442, sinon il reste pareillement performant. Grâce à ce petit test qui se rapproche plus de la réalité, nous pouvons voir que nos modèles s'améliorent petit à petit, signifiant que nos pistes d'améliorations sont bonnes.

1.3 Test de notre modèle sur des descriptions inventées

Maintenant simplement pour tester notre modèle de manière plus brute, voici quelques descriptions écrites par Gemini et réécrites par nous-mêmes pour voir comment le modèle se comporte :

Description	Score prédit	Vecteur prédit	Commentaire
An unauthenticated attacker within Bluetooth range can bypass the pairing PIN of the SmartBrew device to alter the thermal threshold, permanently bricking the hardware	8.1	AV:A/AC:L/PR:N/UI:N/ S:U/C:N/I:H/A:H	Bonne prédiction de l'attack vector en « Adjacent »
A logic flaw in the cloud tenant API allows an analyst with read-only permissions to forge a JWT token and grant themselves global administrator rights	8.8	AV:N/AC:L/PR:L/UI:N/ S:U/C:H/I:H/A:H	Le privilege required est bien en « Low » (read only permission)
A physical button on the digital smart lock, when held for 30 seconds, resets the device to factory defaults without requiring the current master pin, allowing a physical attacker to assume full administrative control	6.8	AV:P/AC:L/PR:N/UI:N/ S:U/C:H/I:H/A:H	Bonne prédiction de l'attack vector en « Physical »
Improper input validation in a local log file analysis tool allows a local authenticated user to create a malicious log entry that, when parsed, leads to a buffer overflow and subsequent local privilege escalation to root	7.8	AV:L/AC:L/PR:L/UI:N/ S:U/C:H/I:H/A:H	Bonne prédiction de l'attack vector en « Local »
With only sending weird empty constructor, user can rce on any nextjs app and extract ram data like credentials and can potentially gain root access to the server	8.8	AV:N/AC:L/PR:L/UI:N/ S:U/C:H/I:H/A:H	Le modèle comprend le terme RCE (remote code execution) (AV:N), il comprend que l'attaque est simple (AC:L et PR:L) et qu'elle est dangereuse (I:H/A:H)

Tableau 3. – Test de notre modèle Only Java sur des descriptions simples

1.4 Conclusion

En conclusion, le modèle est très performant. Nous pourrions encore l'améliorer, en utilisant par exemple de la Data Augmentation (augmenter artificiellement le jeu de données, surtout pour le « only java »), en utilisant d'autres prétraitements (regarder la sémantique des mots et déterminer leur importance) ou bien en équilibrant parfaitement nos classes pour le rendre le plus universel possible. Cependant, au vu du temps que cela peut demander pour seulement quelques pourcents en plus, cela n'est plus une priorité.

1.5 Amélioration du code Google Colab pour la réutilisation

Le code python dédié à l'entraînement a été modifié afin d'être le plus modulaire possible. Les constantes globales sont isolées dans une cellule dédiée en début de notebook, afin de contrôler l'entraînement de manière claire et intuitive, notamment pour la reproductibilité. Ci-dessous les constantes pour l'instant disponibles :

```
## Hyperparamètres
EPOCHS = 10
BATCH_SIZE = 64

### Learning rate => base : 2e-5 | only java : 5e-6 | optuna : 4.846164102597336e-05
LEARNING_RATE = 5e-6

### Weight decay => optuna : 0.04315610182285453
WEIGHT_DECAY = 0.04315610182285453

### Dropout rate => base : 0.3 | optuna : 0.39227009458135786
DROPOUT_RATE = 0.39227009458135786

## Modèle
### Pour un modèle déjà entraîné
BASELINE = False
LOAD_PATH = "/content/drive/MyDrive/cvss_model.pt"
PRETRAINED_MODEL = "distilbert-base-uncased"

### Sauvegarde
SAVE_PATH = "/content/drive/MyDrive/cvss_model"

## Dataset
DATASET_PATH = "/content/drive/MyDrive/dataset_cvss_java.csv"
DATASET_TEST_SIZE = 0.2

### Taille du dataset => base : 50_000 | full : 187_034 | only java : 8630
DATASET_LENGTH = 8630
```

Liste 1. – Constantes disponibles sur le notebook

2 Les tests sur les différents sources de l'agrégateur

Pour faire des tests nous avons cree un fichier benchmark.py. Le protocole de test s'appuie sur :

- Un fixture fixe de CPE : 24 identifiants de produits (CPE) critiques (Log4j, Spring, Jackson, Tomcat, etc.).
- Indicateurs mesurés : Temps de réponse (latence), taux de couverture, détection des versions affectées et précision statistique (F1 Score).
- Consensus (Ground Truth) : Une vulnérabilité est considérée comme « réelle » si elle est confirmée par au moins deux sources parmi EUVD, OSV, NVD et GitHub.

2.1 Résumé des performances par source

Ce tableau présente la couverture des sources, le taux de signalement et les statistiques de latence (en millisecondes).

Source	Santé	Couver- ture	Signalé	CPEs Hit	P50 Lat.	P95 Lat.
EUVD	UP	53.8%	44.6%	14	214 ms	462 ms
OSV	UP	88.5%	100.0%	23	465 ms	1481 ms
NVD	UP	50.0%	100.0%	13	1282 ms	1851 ms
GitHub	UP	88.5%	37.7%	23	430 ms	714 ms
JVN	UP	0.1%	0.01%	0	111 ms	126 ms

La latence de JVN est extrêmement faible, mais cela s'explique par une absence quasi-totale de traitement de données utiles, comme le montre le taux de couverture.

2.2 Précision et Rappel (Consensus « Soft Ground Truth »)

L'analyse ci-dessous compare chaque source à une vérité terrain définie par un consensus d'au moins deux sources parmi {EUVD, OSV, NVD, GitHub}.

Source	TP	FP	FN	Précision	Rappel	F1 Score
EUVD	104	11	135	90.4%	43.5%	58.8%
OSV	198	1	41	99.5%	82.8%	90.4%
NVD	159	19	80	89.3%	66.5%	76.3%
GitHub	198	5	41	97.5%	82.8%	89.6%
JVN	0	2	239	0.0%	0.0%	0.0%

- Vrais Positifs (TP) : CVE confirmée par au moins une autre source.
- Faux Positifs (FP) : CVE isolée sans aucune corroboration.
- Faux Négatifs (FN) : CVE présente dans le consensus mais ignorée par la source.
- F1 Score : Moyenne harmonique entre la précision et le rappel.

2.3 Conclusion de l'analyse

À la suite de cette analyse, les décisions suivantes ont été prises concernant l'intégration des flux de données :

- Maintien de OSV et GitHub : Ces sources affichent le meilleur équilibre entre couverture (88.5%) et précision (>97%).
- Conservation de NVD : Malgré une latence élevée (P50 > 1s), elle reste une base de référence pour le consensus.
- Abandon de la source JVN : L'analyse des données montre que la source JVN est totalement inefficace pour notre périmètre de test. Avec un F1 Score de 0% et une incapacité totale à identifier les vulnérabilités du consensus (239 faux négatifs), cette source n'apporte aucune valeur ajoutée. Sa faible latence n'est que le résultat d'un manque de traitement effectif. En conséquence, JVN a été retiré de la pipeline de production.

Le modèle entraîné nous fournissant des résultats très concluants et fiables, la perte d'une source n'est pas impactant, et le retrait des difficultés générées par le traitement des données de réponse de la JVN est le bienvenu.

3 Simplification du code frontend

Le code frontend a été fortement simplifié et la séparation des responsabilités améliorée, car beaucoup de code de tests était présent. Cela va nous permettre de finaliser la mise en production de la solution, avec la correction des bugs restants

4 Objectif pour la séance suivante

- Réduire au maximum les faux positifs et vrai négatifs
- Finir l'intégration de la pipeline
- Fix les derniers bugs (score CVSS à 0,0, ...)
- (Si nous avons le temps) : Moderniser l'interface graphique

5 **Annexe**

1. https://docs.google.com/document/d/11eCs1nqLkFaR1OKISUttD2aPfVNNbwsUF24GuN8RB_w/edit?usp=sharing
2. <https://urlix.me/cvss-colab-code>
3. <https://urlix.me/cvss-models>